

Signed Rank Test - Examples in R

B.A. Hartlaub and A.S. Wagaman

Signed Rank Test Examples

The data sets used in this set of examples were used previously in the Sign test Examples and Intro to R files, if more context is needed for them.

Example 1 - Field Worker Mileage

Story: Do flexible work schedules reduce the demand for resources? The Lake County, Illinois, Health Department experimented with a flexible four-day workweek. For a year, the department recorded the mileage (and other variables) driven by 11 field workers on an ordinary five-day workweek. Then it changed to a flexible four-day workweek and recorded mileage (and other variables) for a year for the same workers in that setting.

We want to know if the four-day workweek resulted in a decrease in typical mileage.

```
day5 <- c(2798, 7724, 7505, 838, 4592, 8107, 1228, 8718, 1097, 8089, 3807)
day4 <- c(2914, 6112, 6177, 1102, 3281, 4997, 1695, 6606, 1063, 6392, 3362)
ds <- cbind(day5, day4)
ds <- data.frame(ds)
summary(ds)
```

```
##      day5      day4
## Min.   : 838   Min.   :1063
## 1st Qu.:2013   1st Qu.:2304
## Median :4592   Median :3362
## Mean   :4955   Mean    :3973
## 3rd Qu.:7906   3rd Qu.:6144
## Max.   :8718   Max.    :6606
```

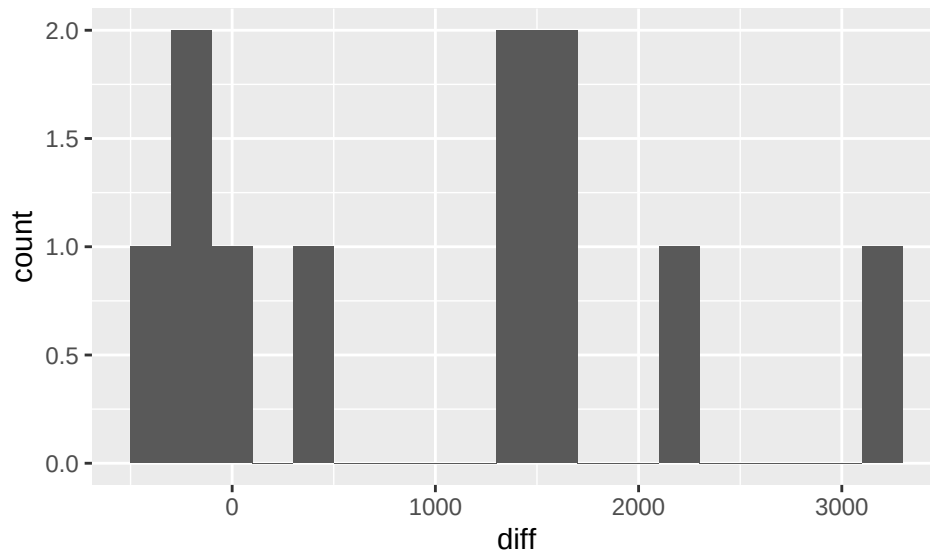
We really want to focus on the differences.

```
ds <- mutate(ds, diff = day5 - day4) # adds to data set
favstats(~ diff, data = ds)
```

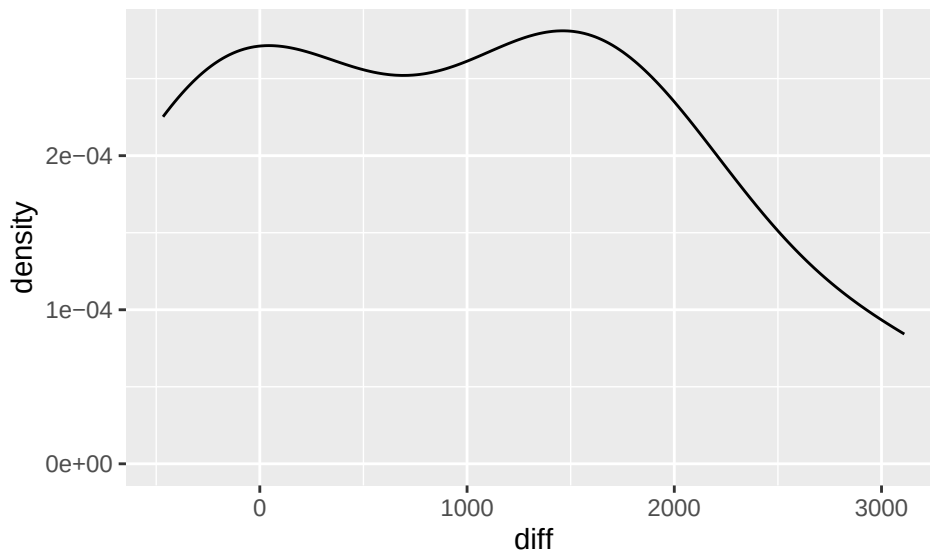
```
##   min  Q1 median    Q3  max mean      sd  n missing
## -467 -41  1311 1654.5 3110  982 1139.568 11      0
```

Some graphical displays:

```
ggplot(aes(x = diff), data = ds) +
  geom_histogram(binwidth = 200)
```



```
ggplot(aes(x = diff), data = ds) +  
  geom_density()
```



Clearly the differences are not normally distributed, and there are only 11 of them.

Now we apply the signed rank test procedure using the *stats* package, which is automatically loaded in R.

```
with(ds, wilcox.test(diff, alt = "greater"))
```

```
##  
## Wilcoxon signed rank exact test  
##  
## data: diff  
## V = 56, p-value = 0.021  
## alternative hypothesis: true location is greater than 0
```

IMPORTANT NOTE: You MUST feed the differences to the `wilcox.test` function if you set no other options and want it to run the signed rank procedure. If you feed it two separate vectors instead, it will

compute the rank sum test used for testing the equality of two medians for two populations (covered later in Hollander, Wolfe, and Chicken).

Alternatively, you can feed it the two vectors, IF you set the paired option to be TRUE. This would look like:

```
# wilcox.test(ds$day5, ds$day4, alt = "greater", paired = TRUE) # base R
```

```
with(ds, wilcox.test(day5, day4, alt = "greater", paired = TRUE))
```

```
##
## Wilcoxon signed rank exact test
##
## data: day5 and day4
## V = 56, p-value = 0.021
## alternative hypothesis: true location shift is greater than 0
```

Note that the notation for the test statistic in R is V instead of $T+$. The provided p-value is 0.021.

We can verify this using `psignrank`, which gives cumulative probabilities for the signed rank test statistic's distribution, but we have to be careful because the distribution is discrete. The help menu states that when `lower.tail = FALSE`, $P(X > x)$ is computed. Thus, if we used $V = 56$, we would get $P(V > 56)$, which is not what we want. We want the probability of getting our test stat of 56 or larger, not just larger than 56. So we have to adjust down one, i.e. we need to find $P(V > 55) = P(V \geq 56)$. You will often have to make this adjustment for upper-tailed tests (but it is not necessary for lower-tailed tests - can you figure out why?). Thus, the correct p-value is:

```
psignrank(55, 11, lower.tail = FALSE)
```

```
## [1] 0.02099609
```

This matches the R output from the test.

If you did a rejection region approach, using $\alpha = 0.05$, you'd find the cutoff using:

```
qsignrank(0.95, 11) #because its an upper-tailed test the input is 0.95, not 0.05
```

```
## [1] 52
```

Now, we want α to be at most 0.05, so we need to check. Using 52 and saying reject if $T+$ is ≥ 52 is going to give us an α of:

```
psignrank(51, 11, lower.tail = FALSE)
```

```
## [1] 0.05078125
```

So, it's a little more than the 0.05 cutoff. If you had to be 0.05 or less, you'd go to 53 as the cutoff, in which case the α would drop to:

```
psignrank(52, 11, lower.tail = FALSE)
```

```
## [1] 0.04150391
```

This is the closest α we can get to 0.05 without going over.

Example 2 - Bleeding Times

```
data("bleedingtime")
```

We want to test to see if a 600-mg dose of aspirin has no effect on bleeding time versus the alternative that it typically leads to an increase in the bleeding time.

We can remind ourselves about the structure of the data.

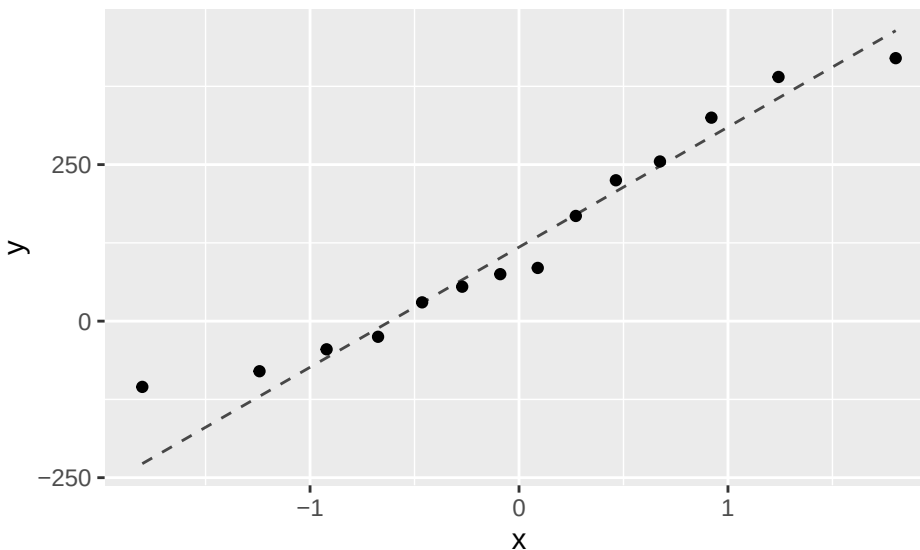
```
glimpse(bleedingtime)
```

```
## Rows: 14
## Columns: 3
## $ subject <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14
## $ xi      <int> 270, 150, 270, 420, 202, 255, 220, 305, 210, 240, 300, 30~
## $ yi      <int> 525, 570, 190, 395, 370, 210, 490, 250, 360, 285, 630, 385, 19~
```

Note that xi is the bleeding time (in seconds) before ingestion of 600 mg of aspirin and yi is the bleeding time (in seconds) 2 hours after administration of the aspirin.

```
# form the differences
bleedingtime$diff <- bleedingtime$yi - bleedingtime$xi # base R
# equivalent to
bleedingtime <- mutate(bleedingtime, diff = yi - xi)
# you could consider using rename to adjust names of xi and yi as well

# what does the distribution look like?
gf_qq(~ diff, data = bleedingtime) %>% gf_qqline()
```



```
with(bleedingtime, wilcox.test(diff, alt = "greater"))
```

```
##
## Wilcoxon signed rank exact test
##
## data: diff
## V = 87, p-value = 0.01477
## alternative hypothesis: true location is greater than 0
```

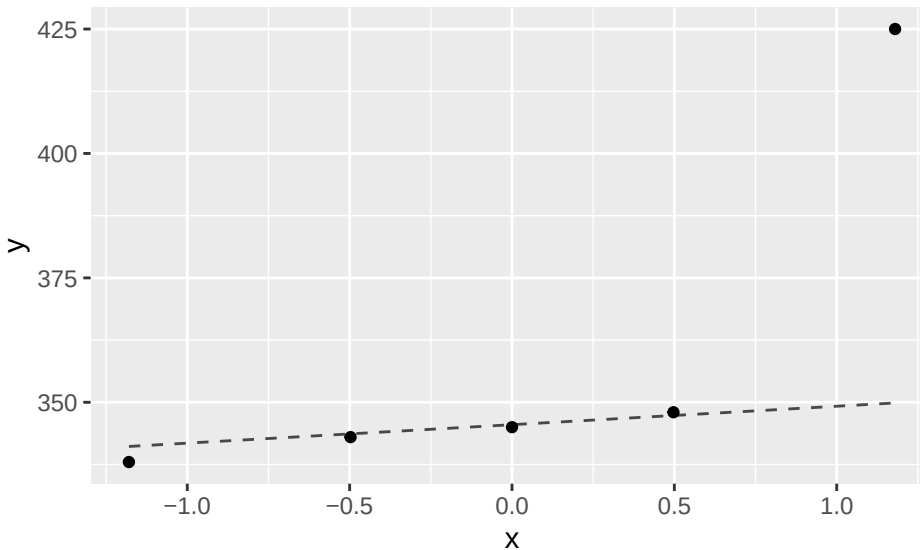
Compare the results of the Wilcoxon signed rank test performed above with the results obtained for the sign test and the paired t test.

Example 3 - Pine Tree Height Differences

In this example, pine trees were planted in 1990 and various measurements taken of them in 1996 and 1997.

We know the pine trees grew from their planting in 1990 until the observations in 1997. But how much? We look at a small sample of trees to understand how the Walsh averages are constructed.

```
data(Pines) # from the Stat2Data package
Pines2 <- slice(Pines, 1:7) # if you look at Pines2 at this stage, you'll see 2 NAs
Pines2 <- Pines2 %>% drop_na() # now we have 5 observations
Pines2 <- Pines2 %>% mutate(diff = Hgt97 - Hgt90) # add the differences in heights
# all three of these commands could be chained together
gf_qq(~ diff, data = Pines2) %>% gf_qqline()
```



We can easily see all differences were positive. Indeed, the test shows we have evidence to conclude that the trees were taller in 1997 than in 1990. We logically already knew that, so our focus is on how much taller the trees are. In other words, we want to obtain a confidence interval.

```
with(Pines2, wilcox.test(diff, alternative = "greater"))
```

```
##
## Wilcoxon signed rank exact test
##
## data: diff
## V = 15, p-value = 0.03125
## alternative hypothesis: true location is greater than 0
```

Now, we aim to get the point estimate and CI. The function `owa` computes ordered Walsh averages. You need the NSM3 package loaded for this to work. The function is set up as `owa(x,y)` and we subtract as `y-x`. Thus, to work with positive differences, we have to run:

```
# currently, these are stored as separate variables
with(Pines2, owa(Hgt90, Hgt97))
```

```
## $owa
## [1] 338.0 340.5 341.5 343.0 343.0 344.0 345.0 345.5 346.5 348.0 381.5 384.0
## [13] 385.0 386.5 425.0
##
## $h.l
## [1] 345.5
```

`h.l` in the output is the point estimate of 345.5.

We can do the computations to find out which Walsh Averages we need for the CI, and pull from the generated list, or we can use the `wilcox.test` function to generate a CI. Bear in mind that the CI from this function

will only match the CI from Hollander, Wolfe, and Chicken if exact p-values are possible (no ties). If there are ties, this function uses a normal approximation, like the large-sample procedure in the textbook.

```
with(Pines2, wilcox.test(diff, alternative = "two.sided",
                        conf.int = TRUE, conf.level = 0.95))
```

```
##
## Wilcoxon signed rank exact test
##
## data: diff
## V = 15, p-value = 0.0625
## alternative hypothesis: true location is not equal to 0
## 95 percent confidence interval:
## 338 425
## sample estimates:
## (pseudo)median
## 345.5
```

Note that we swapped back to a two-sided alternative in order to get a CI. Using a one-sided procedure would have given a confidence bound.

If we wanted to do this by manually computing the C_α value, here is how that would work. Remember the idea is we get all the Walsh averages, then sort them, and pull out the averages at appropriate positions based on the C_α value to get the interval.

The formula for C_α is

$$C_\alpha = \frac{n(n+1)}{2} + 1 - t_{\alpha/2},$$

where $t_{\alpha/2}$ is the upper $(\alpha/2)$ th percentile of the null distribution of $T+$. Finding C_α therefore takes multiple steps.

First, we need to choose a significance level. We are using 95%, so alpha is 0.05. We then look up the upper alpha/2 critical value from the distribution of $T+$ using `qsignrank`. Since critical values like this are often given * for notation, we call this `tstar` for our reference.

```
tstar <- qsignrank(0.975, 5); tstar # specify upper-tail cutoff probability and n
```

```
## [1] 15
```

We then use this value as the t value (remember it's not actually a t distribution) in the C_α computation to get:

```
n <- 5
Calpha <- (n*(n+1)/2)+1-tstar; Calpha
```

```
## [1] 1
```

This gives us the position among the sorted Walsh Averages for the lower confidence limit. For the upper limit, the position is $M + 1 - C_\alpha$. Thus, we need to determine what M is, and M is $n(n+1)/2$, and then compute the upper position.

```
M <- n*(n+1)/2; M
```

```
## [1] 15
```

```
M+1-Calpha
```

```
## [1] 15
```

```
Calpha; M+1-Calpha
```

```
## [1] 1
```

```
## [1] 15
```

So the interval goes from the first Walsh Average to the 15th Walsh Average. Looking at our values from the owa output, we could get this from just reading it or by pulling it out like this:

```
with(Pines2, owa(Hgt90, Hgt97)$owa[c(Calpha, M+1-Calpha)])
```

```
## [1] 338 425
```

Basically here, we ran owa, and knew we wanted some ordered Walsh averages from the output. Then we used vector notation to pull out the 2 values we wanted. This code could all be combined (just change the variable names, alpha, and n) like this:

```
n <- 5
alpha <- 0.05
tstar <- qsignrank((1-(alpha/2)), n); tstar
```

```
## [1] 15
```

```
Calpha <- (n*(n+1)/2)+1-tstar; Calpha # position of lower bound
```

```
## [1] 1
```

```
M <- n*(n+1)/2; M
```

```
## [1] 15
```

```
M+1-Calpha # position of upper bound
```

```
## [1] 15
```

```
with(Pines2, owa(Hgt90, Hgt97)$owa[c(Calpha, M+1-Calpha)])
```

```
## [1] 338 425
```

Again, the wilcox.test function will give you the CI, but the part above shows exactly how it is obtained.

One remaining issue is that the CI may have different coverage than you were expecting; again, due to the discrete nature of these underlying distributions. Here, we specified that we wanted $\alpha = 0.05$, so we wanted a 95% CI. What % did we actually get?

```
n <- 5
alpha <- 0.05
tstar <- qsignrank((1-(alpha/2)), n); tstar
```

```
## [1] 15
```

```
2*(1 - psignrank(tstar-1, n)) # shift down 1 is because of upper-tail computation and
```

```
## [1] 0.0625
```

```
# wanting P(Tplus >= tstar), not P(Tplus > tstar)
1-2*(1-psignrank(tstar-1, n)) # conf level - can convert to a percentage
```

```
## [1] 0.9375
```

So, we actually had a 93.75% CI. Losing confidence is not great, so you'd often just want to move up to the first level higher than 95%. To do that, take the tstar and add 1 (in the case with no ties). In that case,

```

n <- 5
alpha <- 0.05
# tstar is shifted up 1 because we know there is discrete prob here,
# so level is lower than desired
tstar <- qsignrank((1-(alpha/2)), n) + 1; tstar

## [1] 16

# shift down 1 is because of upper-tail computation and wanting P(Tplus >= tstar),
# not P(Tplus>tstar); alpha
2*(1-psignrank(tstar-1, n))

## [1] 0

1-2*(1-psignrank(tstar-1, n)) # conf level - can convert to a percentage

## [1] 1

```

For this example, that's the entire range of values (100%, which isn't helpful). So, we'd probably just report the 93.75% CI, and make sure we specified the level.

Example 4 - Lab Processing Times

We can apply signed rank test procedures when the data are not paired. These are processing times for lab samples during the pandemic. A recent change in testing procedures has led lab staff to ask for more funding as they believe processing times have increased beyond 18 hours per sample. This is a random sample of recent processing times.

```

times <- c(20, 19, 17, 14, 20, 17, 22, 20, 14, 16, 23, 25)
summary(times)

```

```

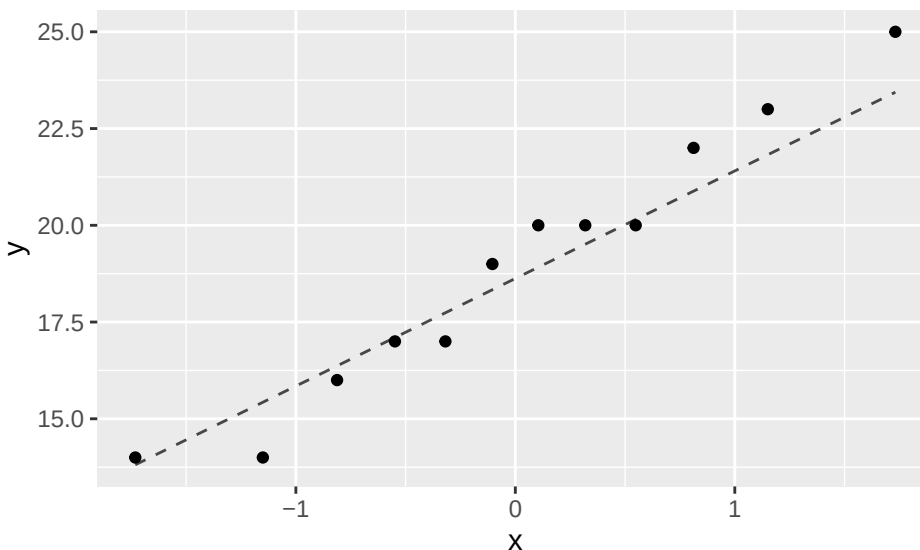
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    14.00  16.75   19.50   18.92  20.50   25.00

```

```

gf_qq(~ times) %>% gf_qqline()

```



Running the Signed rank test here is also fairly simple. We specify $\mu = 18$ to provide the value in our null hypothesis (this could be done even with 2 samples). Note that we don't need to specify the paired option because this is just one sample.


```
wilcox.test(times, mu = 18, alt = "greater")
```

```
## Warning in wilcox.test.default(times, mu = 18, alt = "greater"): cannot compute
## exact p-value with ties
```

```
##
```

```
## Wilcoxon signed rank test with continuity correction
```

```
##
```

```
## data: times
```

```
## V = 50.5, p-value = 0.1924
```

```
## alternative hypothesis: true location is greater than 18
```

The output provided matches our test statistic. Note that you get a warning that the exact p-value cannot be computed due to ties. Due to the ties, trying to get the p-value is also tricky. With no ties, only integers are allowed for values of the statistic, but that changes with ties.

The reported p-value is 0.1924. We can try to figure this out ourselves:

```
psignrank(50.5, 12, lower.tail = FALSE) #  $P(X > 50.5) = P(X > 50) = P(X \geq 51)$ 
```

```
## [1] 0.1696777
```

This gives 0.1697 as the p-value. So, where is the 0.1924 coming from? In the help menu for `wilcox.test`, we learn “By default (if exact is not specified), an exact p-value is computed if the samples contain less than 50 finite values and there are no ties. Otherwise, a normal approximation is used.”

So, the 0.1924 may be from the normal approximation because we have ties in the times. Let’s check that out! Note in order to use the adjusted variance, we have to count numbers of ties, etc. There are three sets of ties, 2 in pairs, 1 triple. All untied observations count as groups of size 1, and end up contributing 0 to the sum (check the formula). So we get:

```
n <- 12
mean <- n*(n+1)/4
varunadjpiece <- n*(n+1)*(2*n+1)
varadjpiece <- (-0.5)*(2*1*3 + 2*1*3 + 3*2*4)
varadj <- (varunadjpiece + varadjpiece)/24
varunadj <- varunadjpiece/24
Z <- (50.5-mean)/sqrt(varadj)
1-pnorm(Z)
```

```
## [1] 0.1829385
```

```
Z2 <- (50.5-mean)/sqrt(varunadj)
1-pnorm(Z2)
```

```
## [1] 0.1834928
```

When we perform the variance adjustment described in the Hollander, Wolfe, and Chicken textbook, we obtain a p-value of 0.1829. When we do not perform the variance adjustment, we obtain 0.1834. What does this tell you about whether or not R is performing the variance adjustment? Note: We do have a small sample size here, but this is using an approximation, where we should be adjusting the variance.

ANSWER

Note: it doesn’t appear that R is performing the variance adjustment, but this situation has a small sample size. Perhaps it is coded to do the adjustment for larger samples. The help file is not clear on this. This is an example of why it is important to check that functions are doing what you expect.

Example 5 - Strontium Levels

This data set is from an exercise in an intro stats textbook. It is convenient for looking at Walsh averages because there are only 5 observations. Five dairy farms are compared in terms of their levels of strontium in 2 percent versus non-fat milk.

```
strontium2 <- c(7.1, 9.9, 11.2, 10.5, 8.8)
strontiumnf <- c(6.4, 5.8, 6.5, 7.7, 6.1)
diff <- strontium2-strontiumnf
diff
```

```
## [1] 0.7 4.1 4.7 2.8 2.7
```

We can easily see all differences (2 percent versus non-fat) were positive. Indeed, the test shows we have evidence to conclude that the levels appear to be higher in 2 percent versus the non-fat milk.

```
wilcox.test(diff, alternative = "greater")
```

```
##
## Wilcoxon signed rank exact test
##
## data: diff
## V = 15, p-value = 0.03125
## alternative hypothesis: true location is greater than 0
```

Now, we aim to get the point estimate and CI. The function `owa` computes ordered Walsh averages. You need the NSM3 package loaded for this to work.

Recall to work with positive differences, we have to watch the order we input the variables:

```
owa(strontiumnf, strontium2)
```

```
## $owa
## [1] 0.70 1.70 1.75 2.40 2.70 2.70 2.75 2.80 3.40 3.45 3.70 3.75 4.10 4.40 4.70
##
## $h.l
## [1] 2.8
```

`h.l` in the output is the point estimate of 2.8.

We can do the computations to find out which Walsh Averages we need for the CI, and pull from the generated list, or we can use the `wilcox.test` function to generate a CI. Bear in mind that the CI from this function will only match the CI from HWC if exact p-values are possible (no ties). If there are ties, this function uses a normal approximation, like the large-sample procedure in the textbook.

```
wilcox.test(diff, alternative = "two.sided", conf.int = TRUE, conf.level = 0.95)
```

```
##
## Wilcoxon signed rank exact test
##
## data: diff
## V = 15, p-value = 0.0625
## alternative hypothesis: true location is not equal to 0
## 95 percent confidence interval:
## 0.7 4.7
## sample estimates:
## (pseudo)median
## 2.8
```

Note that we swapped back to a two-sided alternative in order to get a CI. Using a one-sided procedure would have given a confidence bound.

If we wanted to do this by manually computing the positions of the endpoints, we need the C_α and M values. Here is how that would work (formulas above). First, we need to choose a significance level, and we are using 95%, so alpha is 0.05. We then look up the upper alpha/2 critical value from the distribution using `qsignrank`.

```
tstar <- qsignrank(0.975, 5); tstar # specify upper-tail cutoff probability and n
```

```
## [1] 15
```

We then use this value as the t value (in the book notation, remember it's not actually a t distribution) in the C_α computation to get:

```
n <- 5
Calpha <- (n*(n+1)/2)+1-tstar; Calpha
```

```
## [1] 1
```

The lower bound is then the first Walsh Average and for the upper bound, we need the $M+1-C_\alpha$ position.

```
M <- n*(n+1)/2; M
```

```
## [1] 15
```

```
M+1-Calpha
```

```
## [1] 15
```

So, as in our last example with five observations, the interval goes from the first Walsh average to the 15th Walsh average. Looking at our values from the `owa` output, we could get this from just reading it or by pulling it out like this:

```
owa(strontiumnf, strontium2)$owa[c(Calpha, M+1-Calpha)]
```

```
## [1] 0.7 4.7
```

Basically here, we ran `owa`, knew we wanted some ordered walsh averages from it and used vector notation to pull out the 2 in particular we wanted. This code could all be combined (just change the variable names, alpha, and n) like this:

```
n <- 5
alpha <- 0.05
tstar <- qsignrank((1-(alpha/2)), n); tstar
```

```
## [1] 15
```

```
Calpha <- (n*(n+1)/2)+1-tstar; Calpha # position of lower bound
```

```
## [1] 1
```

```
M <- n*(n+1)/2; M
```

```
## [1] 15
```

```
M+1-Calpha # position of upper bound
```

```
## [1] 15
```

```
owa(strontiumnf, strontium2)$owa[c(Calpha, M+1-Calpha)]
```

```
## [1] 0.7 4.7
```

Again, the `wilcox.test` function will give you the CI, but the part above shows exactly HOW it is obtained.

One remaining issue is that the CI may have different coverage than you were expecting, again, due to the discrete nature of these underlying distributions. Here, we specified that we wanted $\alpha = 0.05$, so we

wanted a 95% CI. What % did we actually get? It's the same as observed above because the n is the same. So, it's a 93.75% CI, just under our desired 95%.

Data Sources

Charles S. Catlin, "Four-day Work Week Improves Environment," Journal of Environmental Health, Denver, 59:7, 1997.

Bleeding times - nsm3data R package - Hollander, Wolfe, and Chicken - Table 3-7.

Pine data from Stat2Data package from Biologists at Kenyon College.

Pandemic sample processing data was simulated.

Strontium levels data from Exercise 16.11 in Devore and Peck's Exploration and Analysis of Data.

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to Nonparametric Statistical Methods (3rd Edition) by Hollander, Wolfe, and Chicken (HWC). Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

This file is best used in conjunction with students reading sections 3.1-3.3, and 3.7 of Hollander, Wolfe, and Chicken.

License

This work is protected by creative commons license CC BY-NC-SA.